

Caso de Estudio

Sistema de Gestión de Incidencias con Enrutamiento Inteligente

Análisis de problemas, decisiones y metodología en Ingeniería de Software

Sergio Duván Blanco Fonseca

Brandon Santiago Forero Alarcón

Sebastián Camilo Romero Páez

Facultad de Ingeniería, Programa de Ingeniería de Sistemas

Ingeniería de Software, Sexto Semestre

Julián Prado

20 de agosto de 2026

Introducción

Este caso de estudio analiza las principales situaciones, problemas y decisiones presentadas durante el desarrollo del Sistema de Gestión de Incidencias con Enrutamiento Inteligente, un proyecto académico desarrollado bajo la metodología Scrum por un equipo de tres integrantes. El análisis se apoya en el Documento de Requisitos de Software (versión 2.0) y en la documentación de arquitectura y avances generada durante el Sprint 1.

Se abordan seis aspectos de la Ingeniería de Software: conocimientos técnicos y metodología, requisitos, calidad, confidencialidad, responsabilidad y privacidad, estructurados en los cuatro puntos solicitados: problemas encontrados, decisiones tomadas, metodologías utilizadas, y errores y consecuencias.

1.Problemas Encontrados

1.1 Dificultades técnicas

- Ambigüedad en las reglas de clasificación: varias categorías del sistema comparten palabras clave (por ejemplo, la palabra "acceso" aparece tanto en la categoría Permisos como en Cuentas y contraseñas). Por esto fue necesario definir reglas de desempate explícitas antes de implementar el algoritmo, de modo que un mismo texto no pudiera clasificarse de dos formas distintas.
- Evaluación de un enfoque alternativo de clasificación mediante un modelo de lenguaje local (Ollama, llama3), realizada de forma previa a la implementación final y posteriormente descartada. Esta prueba, documentada como spike técnico, reveló baja consistencia en las etiquetas generadas para conceptos equivalentes (por ejemplo, "Sistema" y "Sistemas" como categorías separadas para casos similares) y una precisión de solo 3 de 5 casos en la asignación de prioridad, con 2 casos sobre-priorizados sin justificación clara en el texto del ticket. Estos hallazgos fueron el motivo por el cual se descartó este enfoque a favor de un algoritmo de reglas fijas.

1.2 Dificultades en la definición de requisitos

Además de los problemas técnicos, el diseño del sistema exigió resolver con claridad dos preguntas de requisitos antes de poder implementarlas, porque una definición ambigua habría generado comportamientos inconsistentes en el sistema:

- Cómo debía relacionarse la prioridad calculada por el algoritmo de clasificación (según categoría y palabras de urgencia en el texto) con el ajuste adicional que se aplica cuando el solicitante es de nivel VIP o ejecutivo, de forma que ambas reglas trabajaran juntas sin contradecirse.
- Qué acciones dentro del sistema debían considerarse "críticas" y por lo tanto quedar registradas en el módulo de auditoría, evitando un registro incompleto que dejara huecos de trazabilidad, o uno tan extenso que dificultara su revisión.

Nota: estos dos puntos corresponden a decisiones de diseño que el equipo debió resolver de manera anticipada; no está documentado si se presentaron como una dificultad real durante el desarrollo o si se definieron sin contratiempos. Se recomienda confirmarlo con el equipo.

1.3 Dificultades organizacionales

El equipo está compuesto por tres integrantes, dos de ellos asignados por el docente sin experiencia previa confirmada en desarrollo de software, lo cual introduce de forma anticipada un riesgo de desbalance en la carga de trabajo y de concentración del conocimiento técnico crítico en una sola persona (el desarrollador principal).

El corte del Sprint 1 aún no ha finalizado al momento de este análisis, por lo que todavía no es posible determinar si hubo o no retraso frente al plan original de dos semanas. Este punto se debe completar una vez cierre el sprint (ver sección 4).

1.4 Dificultades éticas y de auditabilidad

Un sistema que decide de forma automática el orden en que se atienden los tickets afecta directamente a personas reales dentro de una organización, por lo que sus decisiones deben poder explicarse. El caso más claro de este riesgo se presentó durante la evaluación previa de un modelo de lenguaje para la clasificación (posteriormente descartada): el modelo asignó prioridades distintas a tickets similares sin que existiera una justificación clara en el propio texto del ticket, lo cual habría hecho imposible responder con precisión si un usuario reclamara por qué su solicitud recibió cierta prioridad. Este hallazgo fue una de las razones que llevó a descartar dicho enfoque.

El ajuste de prioridad por nivel del solicitante (VIP/ejecutivo) plantea un reto similar, aunque de distinta naturaleza: es una regla de negocio intencional y no un error del algoritmo, pero si no queda claramente documentada y visible en el sistema, puede percibirse como un trato preferencial injustificado. Por ello es importante que esta regla quede expuesta de forma transparente, y no oculta dentro de la lógica interna del clasificador.

2.Decisiones Tomadas

Decisión	Motivo
Descartar el modelo de lenguaje (Ollama/llama3) y usar un algoritmo determinístico de reglas por palabras clave.	El spike técnico documentó etiquetas inconsistentes y prioridades no reproducibles, inviables para un sistema que necesita agrupar tickets de forma confiable y ser auditable.
Centralizar las reglas de clasificación (palabras clave por categoría) en un solo lugar del código, separadas del motor del algoritmo.	Permite ajustar las reglas sin modificar la lógica de clasificación, reduciendo el riesgo de introducir errores al mantener el sistema.

Decisión	Motivo
Separar la lógica de negocio de las entidades de datos mediante clases de servicio (ServicioAutenticacion, ClasificadorTickets, GestorSLA).	Aplicación del principio de responsabilidad única: permite probar cada pieza de lógica de forma aislada, sin depender de una base de datos completa montada.
Restringir el manejo de contraseñas en texto plano a una única clase (ServicioAutenticacion) y almacenar solo su hash con bcrypt.	Reduce la superficie de exposición de datos sensibles; solo un componente del sistema puede, en algún momento, manejar una contraseña sin procesar.
Usar SQLAlchemy como ORM en lugar de SQL manual.	Evita errores de sintaxis SQL repetidos y mantiene el modelo de datos como código Python versionable y revisable en cada commit.
Dividir el alcance en un MVP obligatorio y extensiones sujetas a tiempo disponible (SLA, auditoría, bloqueo por fuerza bruta, panel de métricas).	Dado el tamaño del equipo (dos integrantes sin experiencia previa) y la duración fija del semestre, se protege la entrega de lo funcionalmente indispensable antes que lo deseable.
Establecer que ninguna tarea de la ruta crítica del proyecto dependa de un integrante distinto al desarrollador principal.	Mitiga el riesgo de bloqueo total del proyecto si alguno de los integrantes sin experiencia previa no logra completar una tarea asignada a tiempo.

3. Metodologías usadas

El equipo adoptó el marco de trabajo Scrum, con sprints de dos semanas, complementado con un enfoque de prototipado evolutivo (spike técnico) para reducir la incertidumbre antes de comprometer decisiones de diseño.

3.1 Elementos de Scrum aplicados

- Roles diferenciados: Product Owner (definido junto al docente), Scrum Master (desarrollador principal) y Dev Team (los tres integrantes).
- Ceremonias: Sprint Planning, Daily Standup asíncrono por chat de equipo, Sprint Review y Sprint Retrospective al cierre de cada sprint.
- Definition of Ready (DoR): cada historia debe tener criterios de aceptación claros, caber en un solo sprint y no depender de una historia inconclusa.
- Definition of Done (DoD): código integrado a la rama principal, probado manualmente contra sus criterios de aceptación, comentado y con documentación actualizada.

3.2 Spike técnico como práctica de reducción de riesgo (Descartada)

Antes de comprometer el diseño del módulo de clasificación, el equipo ejecutó un spike de tiempo acotado evaluando un modelo de lenguaje local como posible alternativa. Se probaron 5 tickets de prueba y se midieron tres criterios objetivos (validez del formato JSON de respuesta, coincidencia con la categoría esperada y tiempo de respuesta). Los resultados llevaron a descartar esta alternativa y a optar por un algoritmo de reglas fijas,

evidenciando que la metodología permitió tomar la decisión de arquitectura con datos medibles en lugar de intuición, evitando construir el sistema completo sobre una base que luego habría que reemplazar.

3.3 Contribución de la metodología a los problemas identificados

- El spike evitó construir todo el sistema de clasificación sobre una base no determinística, detectando el problema antes de que su costo de corrección fuera alto.
- La separación en carpetas de modelos y servicios, visible en la estructura real del repositorio, es la traducción directa a código del principio de responsabilidad única acordado en el diseño, lo que permite evaluar el avance real por componente en cada sprint.
- El seguimiento del Sprint 1 en tres estados (completado, en progreso y pendiente) refleja que el Sprint Backlog se está usando activamente para visibilizar el avance, y no solo como documento formal.

4. Errores y Consecuencias:

DADA LA ETAPA DE DESARROLLO ACTUAL, AUN NO HA HABIDO ERRORES O CONSECUENCIAS SIGNIFICABLES PUESTO QUE AUN SE ESTA EN LA ETAPA INICIAL DEL PROYECTO

Reflexión Individual

POR EL MOMENTO AUN NO

Conclusión

El desarrollo del proyecto muestra decisiones de ingeniería tomadas con criterio explícito y verificable, particularmente el descarte razonado de un enfoque de clasificación basado en inteligencia artificial a favor de un algoritmo determinístico y auditable, y una metodología (Scrum con spike técnico previo) que contribuyó directamente a reducir la incertidumbre antes de comprometer la arquitectura. El punto pendiente de mayor valor para completar este caso de estudio es la documentación honesta de los errores organizacionales y técnicos ocurridos durante el Sprint 1, información que solo el equipo de desarrollo puede aportar con precisión una vez cierre el corte.